

# Peaceful conflict resolution in version control systems

Mathias Rav

Scalgo

---

## Abstract

---

When collaborating on a codebase where several people are working in parallel on different development branches, it often happens that different developers try to edit the same piece of code, which leads to problems when both branches have to be merged into the main codebase.

If git is used to collaborate on the codebase, the first developer to merge the branch has an easy day, and the other developer is met with a *merge conflict*, which is the technical term used when git was unable to automatically integrate the two sets of changes together.

In this article, we present a way of thinking about codebase changes that lets us handle merge conflicts in a novel way that is much less of a hair-pulling frustrating experience. By using this new way of thinking, we then develop a new mechanical way to work with branches that makes it easy to split off “refactoring work” from “feature work”, such that refactors can be merged early, thus reducing the incidence of merge conflicts.

## 1 Introducing the Hammer

Refactors often come up organically while working on a codebase; small or uncontroversial refactors can be done routinely while working on a feature. However, the more refactors there are in a feature branch, and the longer the refactors stay in the feature branch, the more likely it is that a merge conflict rears its ugly head when the feature branch is ready to be merged into the main branch. For this reason, it is preferable to merge refactors early, as much as the software development process allows. There are no standard tools that make it easy to untangle a refactoring from a feature addition when the refactoring is implemented simultaneously with the feature addition.

In this paper, we introduce a git technique, dubbed *the Hammer*, which makes it easy to split refactorings from feature additions as well as to handle conflicts in a git interactive rebase without having to deal with merge conflict markers. This paper presents the Hammer together with three key use cases, namely (1) splitting a commit into a refactoring and a feature addition, (2) moving an arbitrary change from one commit into another, and (3) swapping two commits that have a syntactic conflict. The Hammer relies on the developer to manually redo or undo some work that has already been done as part of existing commits in the branch, and in return, the developer never has to resolve merge conflicts by merge conflict marker soup. Since redoing a bit of programming work utilizes the skills of reading and writing code that developers are trained to do, the Hammer technique exchanges mentally straining work on merge conflict markers for much simpler programming work that is easier on the mind and soul.

► **Definition 1.** *The Hammer: Insert  $A$  and  $A^{-1}$  at some point in a commit sequence, then squash  $A$  up and/or squash  $A^{-1}$  down.*

Here,  $A$  is an arbitrary commit, and  $A^{-1}$  is the commit obtained by running `git revert HEAD` immediately after committing  $A$ . These two commits cancel each other out, meaning they can safely be inserted into a sequence of commits (say, using an interactive git rebase) without introducing merge conflicts. It is pointless to keep both commits in a branch, which is why

the Hammer dictates that one or both of the commits must be squashed. Squashing one of the commits into another commit in the sequence has the effect of splitting off an arbitrary part of that other commit; squashing both  $A$  and  $A^{-1}$  into other commits in the sequence has the effect of moving an arbitrary bit of change from one commit into another. Whereas reordering commits in an interactive rebase can potentially fail due to conflicts, squashing adjacent commits can never cause conflicts.

Let us go through three use cases for the Hammer. Firstly, to split a refactor from a feature addition with the Hammer, we simply delete the new feature from the codebase, store the deletion as a new commit, and immediately revert the deletion of the feature. The commit that removes the feature is squashed into the initial commit, leaving behind a commit that only refactors without adding any feature, and the revert of the deletion of the feature becomes a commit that adds the feature. If we use  $A$  to denote the codebase change of adding the feature, and  $B$  the codebase refactoring, then the initial commit is  $AB$  (a combination of the two changes), and after the feature deletion and revert, the branch contains the three commits:  $AB$ ,  $A^{-1}$ , and  $A$ . Here, the developer had to implement  $A^{-1}$  (the deletion of the new feature) by hand, but the commit for  $A$  was generated automatically using `git revert`. By squashing  $A^{-1}$  into  $AB$ , we obtain two commits, first  $B$  (the refactoring) and then  $A$  (the feature addition). The commit with the change  $B$  can then be merged into the main branch without having to wait for the feature to be fully implemented.

The second use case, moving an arbitrary change from one commit into another, can come up as a part of splitting a refactoring from a feature addition, as it might be the case that the developer failed to delete the entirety of the feature when applying the Hammer in the first use case; the result is two commits, where the first commit is mostly a refactoring with a little bit of the feature-addition work, and the second commit contains the rest of the feature addition. In this case, the Hammer is used in an interactive rebase session to insert two commits  $A$  and  $A^{-1}$  in-between the two existing commits, where  $A$  deletes the remaining parts of the feature addition (done by the developer by hand), and  $A^{-1}$  (made automatically with `git revert`) undoes the deletion of the feature addition;  $A$  is squashed into the first commit and  $A^{-1}$  into the second commit, which effectively moves the last bit of feature addition from the first commit into the second.

The third use case, swapping the order of two arbitrary commits, comes up when a bug is discovered in an early commit in the feature branch, and the bugfix is implemented as the last commit in the feature branch, and an attempt to move the bugfix up in an interactive rebase fails due to a conflict. In this case, the bugfix  $A$  can be swapped with an intermediary conflicting commit  $B$  by inserting two new commits  $A'$  and  $A'^{-1}$  just before  $B$  in the feature branch, where  $A'$  is the same bugfix as  $A$  (implemented by the developer by reading the diff of  $A$  and applying it by hand to the codebase before  $B$ ) and  $A'^{-1}$  is obtained automatically with `git revert`. The result is a sequence of four commits,  $A'$ ,  $A'^{-1}$ ,  $B$ ,  $A$ , where the last three are squashed together to  $A'^{-1}BA = B'$ , which implements  $B$  on a version of the codebase where the bugfix  $A'$  has already been applied.

The full conference talk will spend the first third of the time on introducing merges, merge conflicts, a refresher on git rebase, syntactic vs. semantic conflicts, and the kinds of workflow that inevitably leads to syntactic conflicts; the remaining time is spent on use cases of the Hammer as well as a rundown of the four fundamental ways in which the Hammer can be applied. In practice, the Hammer should be combined with other standard git techniques to split commits into smaller changes, which are also touched upon towards the end, together with some reflections on the situations where the Hammer is not the best choice.